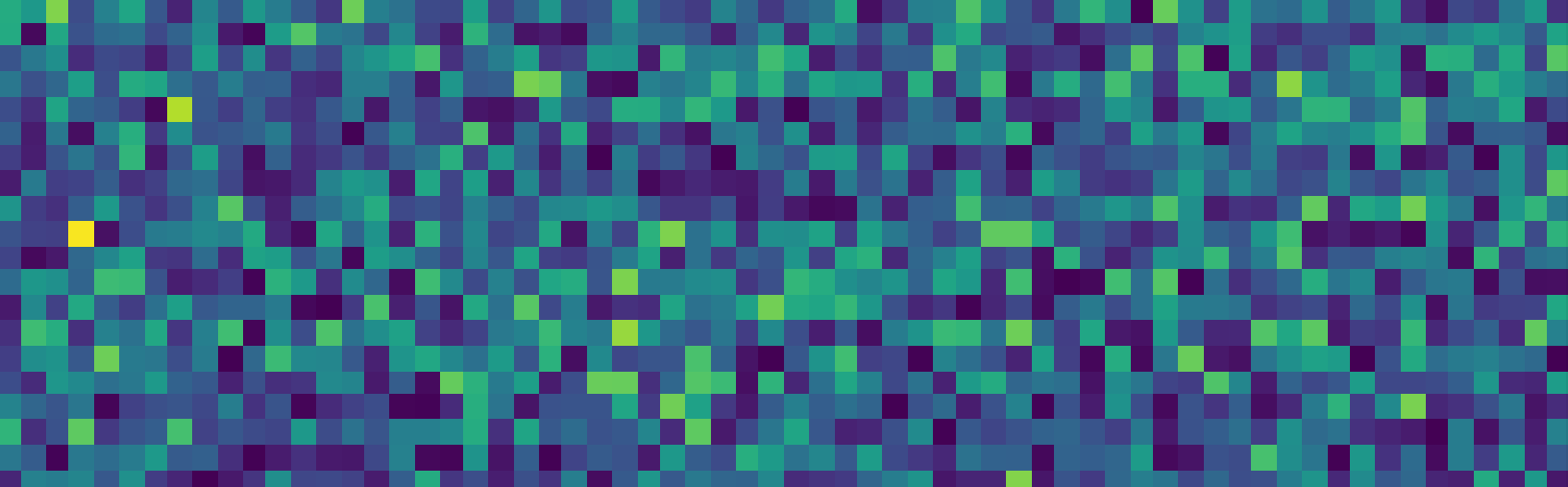




Neural Collapse Pruning

Alex Ge

November 16 2026

MEng Project Report

Supervisor: Vardan Papyan and Stephen Zhang

Table of Contents

1.0 Introduction.....	2
2.0 Methods.....	3
2.1 Converting LayerNorm to RMSNorm.....	3
2.2 Sampling embeddings.....	4
2.3 Applying Invariant Transformations.....	5
2.3.1 Applying rotational transformations.....	5
2.3.2 Applying translation, scale, and permutation transformations.....	9
2.3.3 Applying non-orthogonal transformations.....	12
2.4 Performing OATS compression.....	13
3.0 Results.....	13
3.1 Effect of Rotating Model Weights.....	14
3.2 Effect of Translation, Scale, and Permutation Transformations.....	16
3.4 Effect of Non-orthogonal Transformation.....	17
4.0 Discussion.....	19
References.....	20

1.0 Introduction

In recent years, deep learning techniques have demonstrated transformational capabilities, such as in language generation and reasoning [1]. However, most powerful models today are computationally expensive and require specialized computing resources to run [2]. At the same time, it has been observed that over the course of training, models tend to develop significant internal redundancies, and in some cases, more than half of parameters can be removed without affecting performance [3]. This emergence of redundancies represents an opportunity to improve the computational efficiency of current models, and indeed, much effort has been undertaken in recent years to evaluate various methods of compressing models, such as through pruning [3, 11, 12] and distillation [4].

While model optimization has been an intensive area of research, in parallel, mechanistic interpretability has emerged as a powerful tool to analytically and deterministically uncover the underlying mechanisms that explain how and why models behave the way they do. For example, Pappan [5] identified a phenomenon that occurs during the last phase of training, known as Neural Collapse, where the embedding representations of different classes tend to converge at specific points in high-dimensional space in a way that minimizes within-class variance while maximizing between-class variance. Others have found that parameters and features in deep learning models are highly structured, exhibiting geometrical patterns [6], circular features [7, 8], among other patterns [9].

Given the fact that models contain highly organized structures, there is an opportunity to develop a pruning method that prunes around the structures present in deep learning models. In this report, we look at a method of modifying deep learning networks along computational invariances in a way that does not change the output of the model, yet improves pruning efficiency. Specifically, we will build off of an existing pruning method (OATS) [11], by aligning the internal structures of the model in a way that maximizes sparsity of the embedding representations. In section 2, we go over the method and mathematical formulations behind the method. In section 3, we evaluate the performance of the pruning method described in the report. Finally, in section 4, we discuss the results, the implications, and future work.

2.0 Methods

This section describes the method for finding and applying invariant transformations to a model to improve the effectiveness of the OATS pruning method [11]. The method is summarized in the steps that follow.

1. Convert LayerNorm modules to RMSNorm ([section 2.1](#))
2. Pass a calibration dataset through the model and sample embeddings at each layer ([section 2.2](#))
3. Perform invariant transformations on the model weights ([section 2.3](#)), including:
 - a. Rotational invariances ([section 2.3.1](#))

- b. Translation, scale, and permutation invariances ([section 2.3.2](#)), and
- c. Non-orthogonal invariances ([section 2.3.3](#))
4. Perform OATS on the model weights ([section 2.4](#))

The test model that I used in my experimentation was the Google ViT base model [10], trained on the ImageNet dataset.

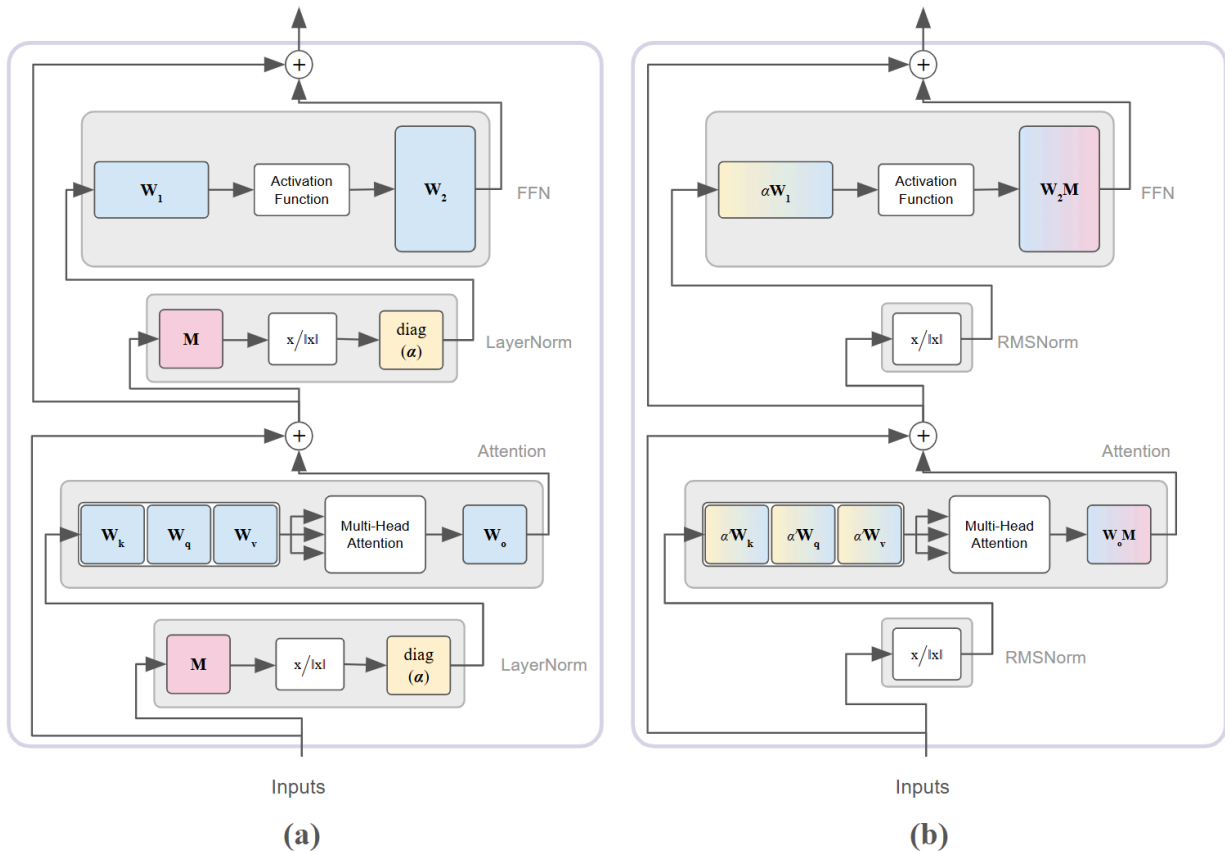


Figure 1 - A model that uses LayerNorm normalization (a) can be converted to RMSNorm (b). Adapted from the SliceGPT paper.

2.1 Converting LayerNorm to RMSNorm

The authors of the SliceGPT paper describe how orthogonal matrices can be applied to the weights of a model due to "computational invariance", allowing the weights to be rotated without affecting the outputs of the model [12]. Notably, this only works on models with RMSNorm modules, so existing LayerNorm modules must first be converted to RMSNorm.

The RMSNorm conversion process is described in detail in the SliceGPT paper, and visualized in Figure 1. In summary, the LayerNorm module can be decomposed into three operations, two of which are absorbed into the surrounding linear transformations:

- The mean subtraction (or centering) operation (represented by the red blocks in Figure 1(a)) is absorbed into the preceding weight matrix (represented by the blue-red gradient blocks in Figure 1(b))
- The scale and shift operation (represented by the yellow blocks in Figure 1(a)) is absorbed into the following weight matrices (represented by the yellow-blue gradient blocks in Figure 1(b))

The remaining unabsorbed block in the original LayerNorm module (represented by the white block between the red and yellow blocks in Figure 1(a)) becomes the new RMSNorm module.

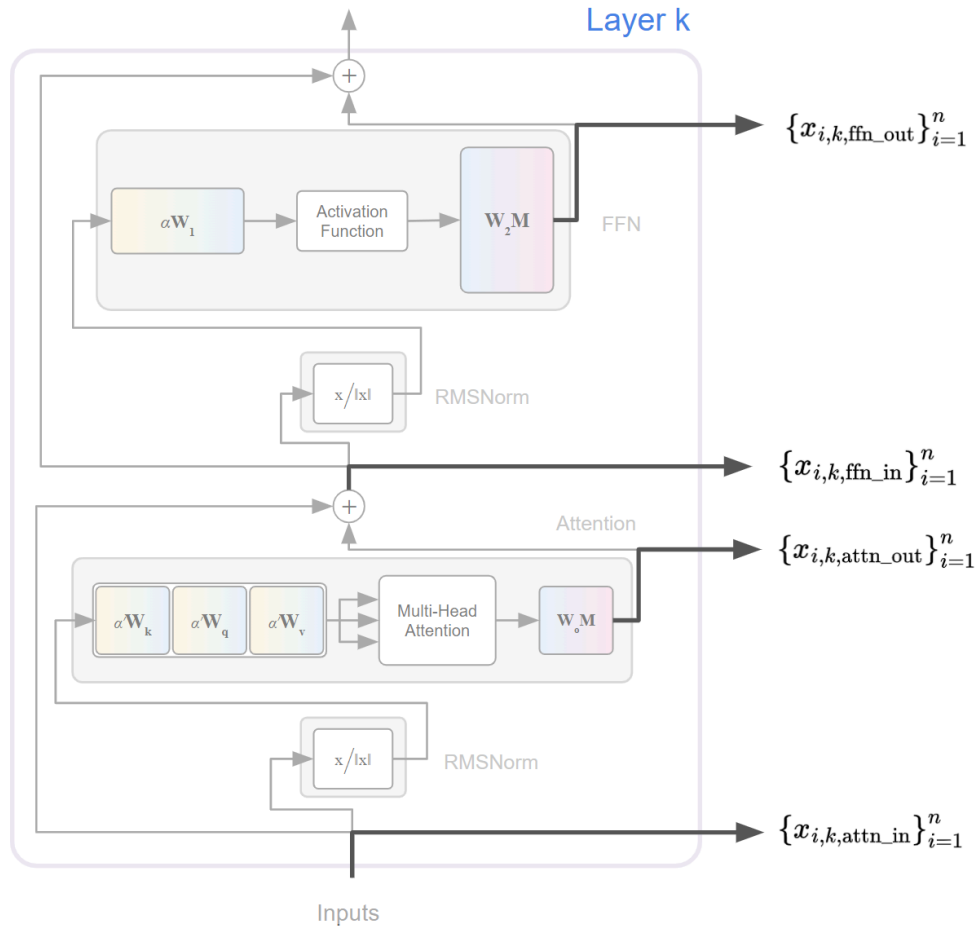


Figure 2 - Location of sampling points in a transformer layer with k representing the layer; n representing the number of samples

2.2 Sampling embeddings

To guide the construction of the transformations to apply to the model, a sample of embeddings at each layer was first collected. To compile a representative sample of embeddings, the validation subset of the ImageNet dataset was used. For each sample in the validation subset, the corresponding embedding representations were collected at four points in each layer of the model, shown in Figure 2. The sampling locations were as follows:

1. Before the RMS Norm of the attention module (*attn_in*)
2. After the attention module, and before the residual stream is added (*attn_out*)
3. After the attention module, and after the residual stream is added (or alternatively, before the RMS norm of the feed forward module) (*ffn_in*)
4. After the feed-forward module, before the residual stream is added (*ffn_out*)

In total, this produced 48 datapoints for each sample (12 layers x 4 collection points) for a total of 2.4 million datapoints (50000 samples x 48 datapoints per sample).

2.3 Applying Invariant Transformations

There exists a range of operations that we can do to the model that do not affect the output. However, these may be of interest to us if they increase the effectiveness of OATS pruning. This section describes the invariant transformations that I explored.

2.3.1 Applying rotational transformations

Rotational invariance is the invariant property used in the SliceGPT and QuaRot pruning methods [2, 3]. Both methods have a different approach for creating the rotation matrix used to transform the model weights. The method outlined below is based on the method used in SliceGPT, and involves rotating the model weights with the principal components of the sampled embeddings.

Calculating the Principal Components

To create the rotation matrix, the first step is to calculate the principal components of the sampled embeddings. In SliceGPT, different rotation matrices are used at each layer of the model, which necessitates an additional matrix operation at each layer to rotate the residual stream. However, in our case, PCA is conducted over sampled embeddings from all layers to create a single "global" rotation matrix. Since the same rotation matrix is used for all layers, no additional matrix operations are needed to rotate the residual streams.

A consequence of conducting PCA over all layers is that the magnitudes of the embeddings at different layers do not necessarily follow the same scale, leading to the embeddings at some layers being over- or under-represented in the principal components. Specifically, I noticed that the magnitudes and variances of the individual features tend to grow as the samples travel

through the model. To visualize this, the boxplots in Figure 3 show the variances of the 768 embedding features at each sampling point throughout the 12 layers of the model. The variances of features appear to consistently increase at later layers.

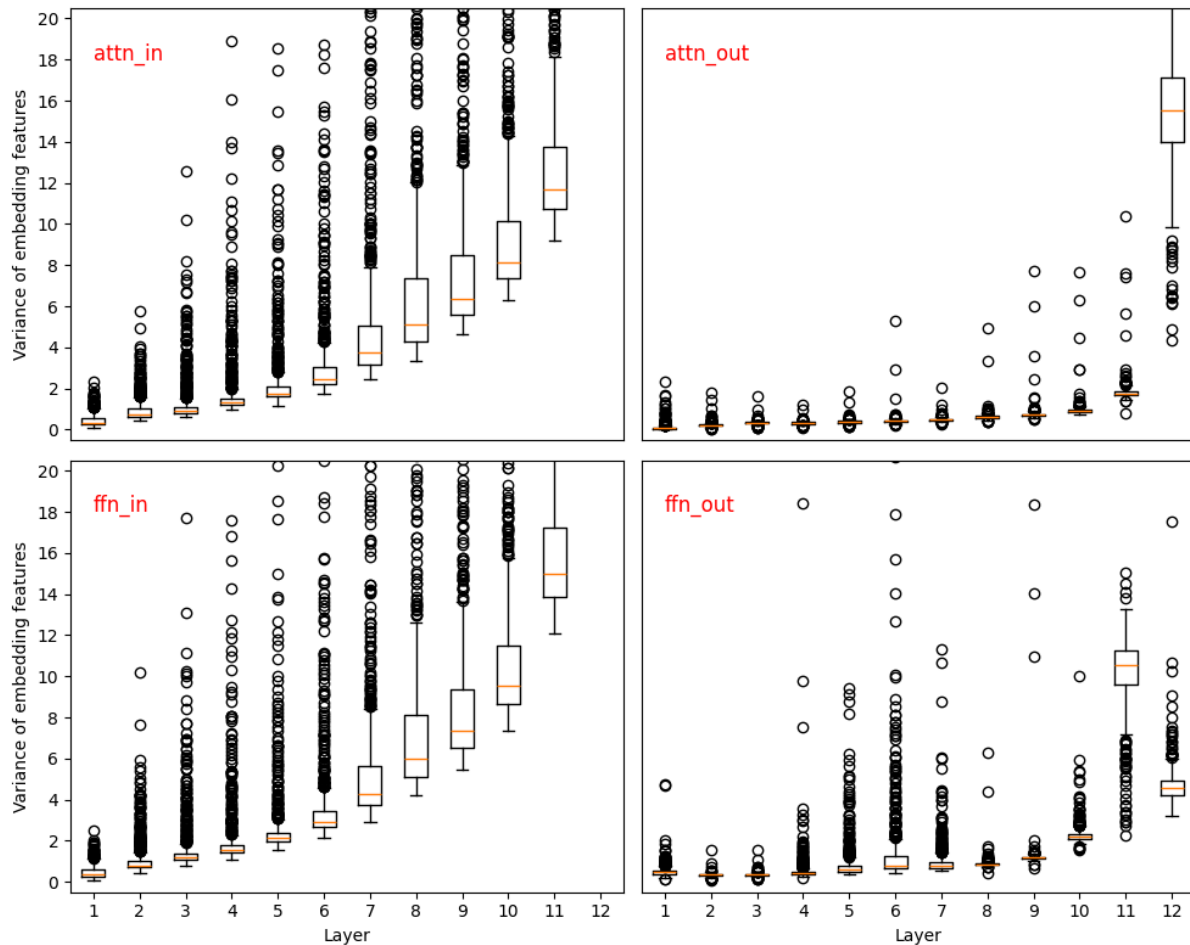


Figure 3 - Box-plot of variances of embedding features at each sampling point

Having larger feature variances at later layers would mean that when PCA is conducted on the embeddings, the principal components are likely to be dominated by the embeddings from the last few layers. Ideally, we want the principal components to be equally representative of embeddings at all layers. To address this, I considered two methods to normalize the contribution of each layer's sampling point: (1) using the trace of the covariance matrix, and (2) using the Frobenius norm of the covariance matrix.

To normalize using the trace of the covariance matrix, we can calculate the covariance matrix of embeddings from each layer and sampling point separately, and normalize the individual

covariance matrices by their trace. Then, the principal components would be calculated using the sum of the covariance matrices from all sampling points. Since the trace of the covariance matrix represents the total variability of the features, by normalizing the covariance matrix with its trace, we can ensure that the contribution of "variation" to the principal components is equal for each sampling point.

Figure 4 shows a diagram that summarizes this process. For the purposes of illustration, the sampled embeddings in layer 1 and 2 are much smaller than layer k, however, by norming the covariance matrix of each layer with its trace, the cumulative covariance matrix is equally representative at all layers.

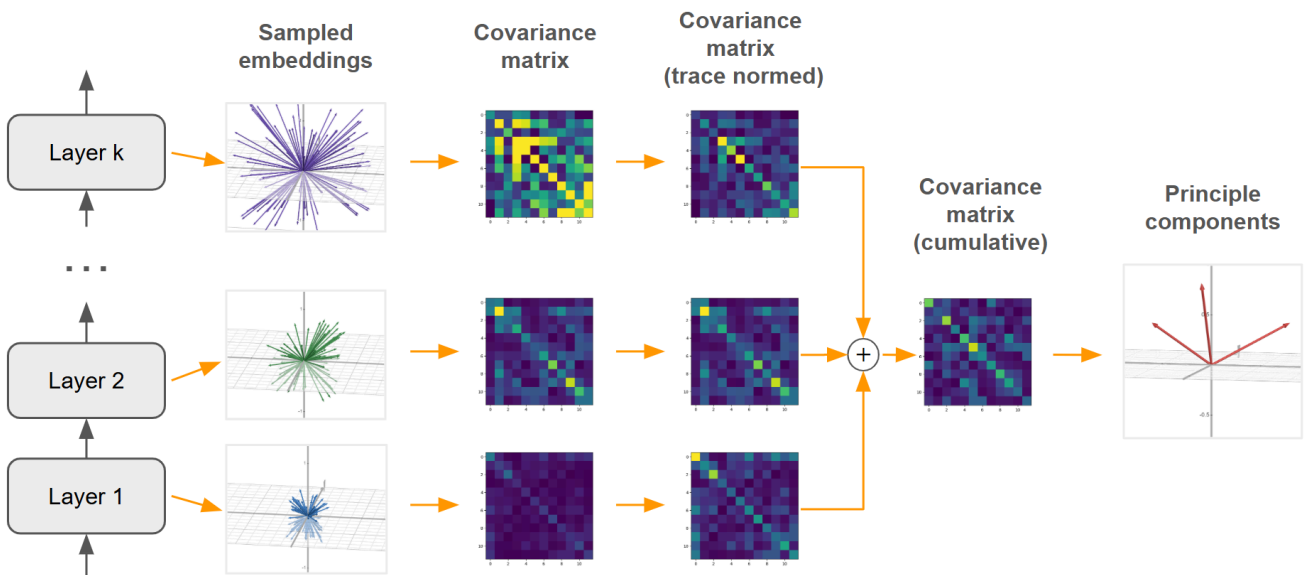


Figure 4 - Normalizing the variation of each sampling point during PCA

Alternatively, instead of normalizing each sampling point's covariance matrices with its trace, we can use the Frobenius norm instead, which ensures that the "magnitude" of each covariance matrix is the same.

Rotating model weights

Once the principal components are calculated, the next step is to apply them to the weight matrices of the model. This is the same method used in the SliceGPT paper, with the exception that we do not need to add a matrix multiplication at the residual stream (as mentioned earlier).

To rotate the model weights, the matrix representing the principal components would be applied to the left side of the weight matrices after a residual stream connection (i.e. the QKV matrices of the attention modules and the in-projection matrix of the feed-forward modules), and the transpose of the matrix would be applied to the right of the matrices before a residual stream connection (i.e. the out-projection matrices of the attention and feed-forward modules).

Figure 5 shows a visualization of the addition of the rotation matrix to the model diagram in the previous section, where Q represents the rotation matrix.

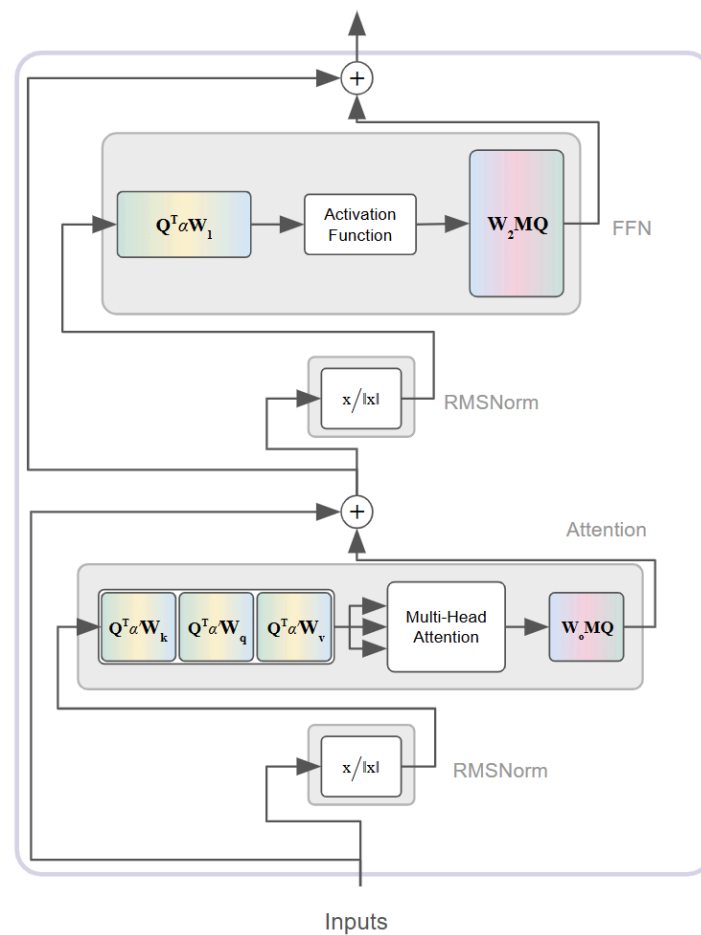


Figure 5 - Applying an orthogonal matrix to the weight matrices of a model.
Adapted from the SliceGPT paper.

2.3.2 Applying translation, scale, and permutation transformations

Translation, scale, and permutation transformations can supplement a rotational transformation of the principal components. Analogously, datasets are commonly standardized before doing PCA, which is done by subtracting the mean vector (which is a translation operation) and

scaling by the standard deviation (which is a scale operation). We can incorporate similar transformations in our model to increase the effectiveness of PCA.

A description of the translation, scale, and permutation transformations is listed in Table 1.

Table 1 - Description of invariant transformations

Operation	Approach	Modifications to Model
Translation	At each sampling point, the mean vector is calculated from the sampled embeddings. The mean vector is subtracted from the bias term on the linear transformations before the sampling point, and added to the bias term after the sampling point. This is equivalent to centering a collection of datapoints before applying PCA (although in our case, the embeddings can be centered with a different mean vector for each sampling point).	(1) An offset vector must be added to the bias of the linear transformation before and after each sampling point. No changes to the residual stream are necessary. (2) Since the RMSNorm module is applied before the offset vector is added to the embeddings, the offset for the linear transformation after the RMSNorm module must be dynamically calculated at inference. Specifically, the mean vector must be divided by the denominator term in the RMSNorm equation.
Scale	At each sampling point, a variance vector is calculated from the sampled embeddings. Using the sampled variances, the model weights are adjusted to scale each embedding feature by its standard deviation. This is equivalent to standardizing each embedding feature's standard deviation before applying PCA.	(1) A scaling/unscaling matrix must be applied to the linear transformations before and after each sampling point. (2) A scaling operation must be added to scale the residual stream at each sampling point. (3) The RMSNorm module must unscale the embedding before calculating the denominator in the RMSNorm equation. (4) If a unique scaling vector is applied at each layer in addition to a rotation matrix, then the residual stream must be unrotated and unscaled based on the transformations from the preceding layer, then scaled and rotated based on the transformations from the following layer. This necessitates at least one additional matrix operation on the residual stream for each layer.
Permutation	At each sampling point, a permutation is applied to order the embedding features from highest variance to lowest variance. The objective would be to align the largest principal components at each sampling point to improve PCA performance. In the future, an improved method can find a permutation that maximizes the similarity of the covariance matrix generated from sampled embeddings at each sampling point (instead of using just the variance).	(1) A permutation/unpermutation matrix must be applied to the linear transformations before and after each sampling point. (2) A permutation operation must be added to permute the residual stream at each sampling point. (3) If a unique permutation is applied at each layer in addition to a rotation matrix, then the residual stream must be unrotated and unpermuted based on the transformations from the preceding layer, then permuted and rotated based on the transformations from the following layer. This necessitates at least one additional matrix operation on the residual stream for each layer.

For the translation operation, we have to add an offset vector to each linear transformation preceding and following a residual stream connection. However, we can choose to either add the offset vector before the linear transformation takes place (which would necessitate a new model parameter) or after the linear transformation (which allows us to combine it with the bias term).

To help visualize this, Figure 6(a) shows the unmodified linear transformations (labelled W_1 and W_2 , respectively) before and after the addition of the residual stream. For the sake of clarity, I have split the affine transformation in each linear operation into matrix multiplication M and the vector addition of the bias β .

Figures 6(b) and 6(c) show the addition of an offset vector. Figure 6(b) adds the offset to the bias of the first linear transformation M_1 , while Figure 6(c) adds the offset before the linear transformation M_1 is applied. Note that in the latter case, since we add the offset before the linear transformation, we need to transform the offset with the inverse of the linear transformation M_1^{-1} .

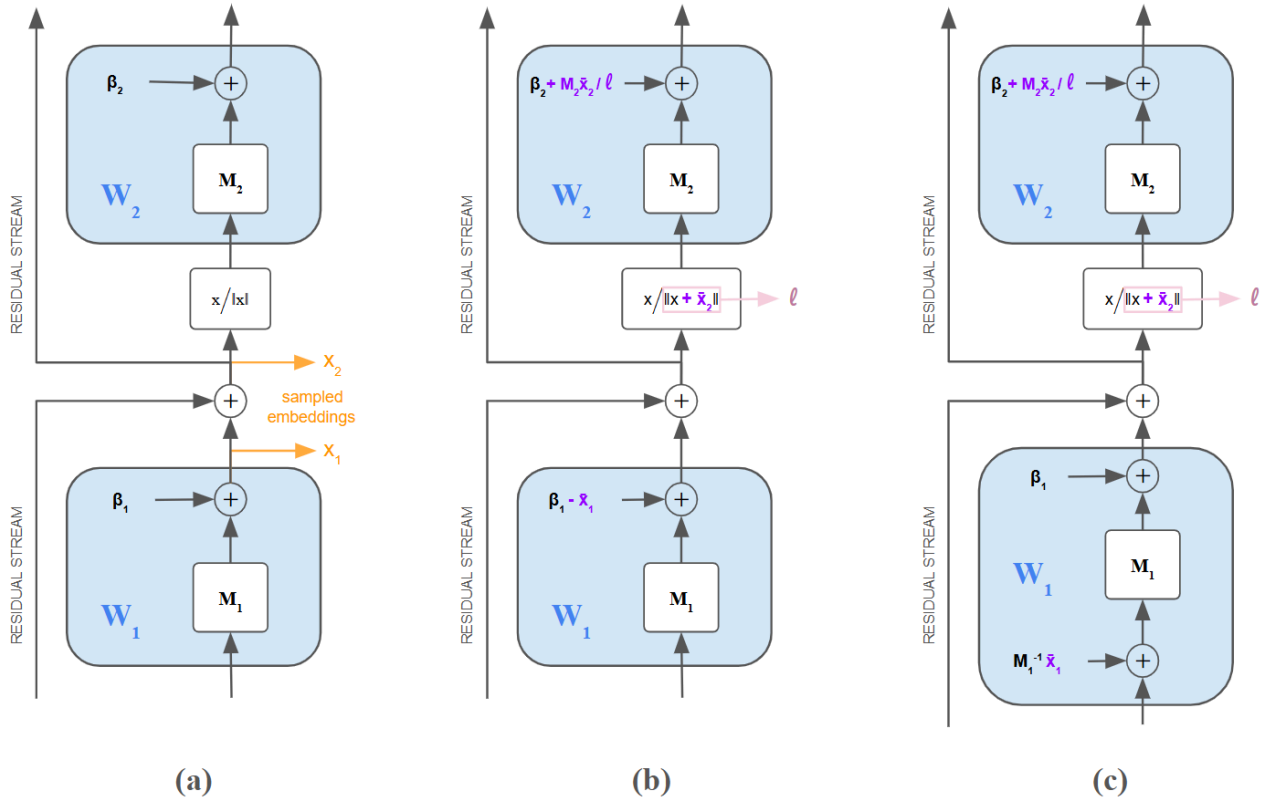


Figure 6 - Application of offset between sampling points to perform invariant translation.

In total, I tried three different translation approaches, as listed below.

- Option A - the offset is added to the bias of both linear transformations (as shown in Figure 6(b))
- Option B - the offset is added before the first linear transformation and after the second linear transformation (as shown in Figure 6(c))
- Option C - the offset is added before the first linear transformation and after the second linear transformation (like option B). In addition, the bias term of the first linear transformation is moved before the linear transformation. The motivation is that moving the bias term before the linear transformation would allow the linear transformation to operate on centered data (with 0 mean) - although it is unclear if this would improve performance.

2.3.3 Applying non-orthogonal transformations

The method to rotate the weight matrices using an orthogonal matrix, outlined in the SliceGPT paper, can be applied to non-orthogonal matrices as well, as long as the matrix is full-rank (i.e. it needs to be invertible).

I explored one such type of matrix, composed of two components: an identity component, plus a low-rank component (I+LR). The mathematical definition of this matrix is given in Equation 1, where I is the identity matrix, U_r is a low-rank matrix (only the first r rows are nonzero), and k is a coefficient from 0 to 1.

Equation 1 - Construction of I+LR matrix (M_{ILR})

$$M_{ilr} = \underbrace{kI}_{\text{Identity matrix}} + \underbrace{(1 - k)}_{\text{Coefficient}} \underbrace{U_r}_{\text{Low-rank matrix}}$$

Compared to an orthogonal matrix, transforming an embedding with an identity and a low-rank component would incur lower memory and computational costs. The benefit of doing this is that we can apply a unique I+LR construction to each layer. As mentioned previously, if we transform the model with different matrix operations at each layer, then we would need to transform the residual stream as well. However, the resource penalty for transforming the residual stream with

an I+LR construction is theoretically significantly lower than using a non-sparse orthogonal matrix.

In my tests, the low-rank component was constructed using the largest principal components at each sampling point. Figure 7 shows new changes using the model figure from [section 1.1](#).

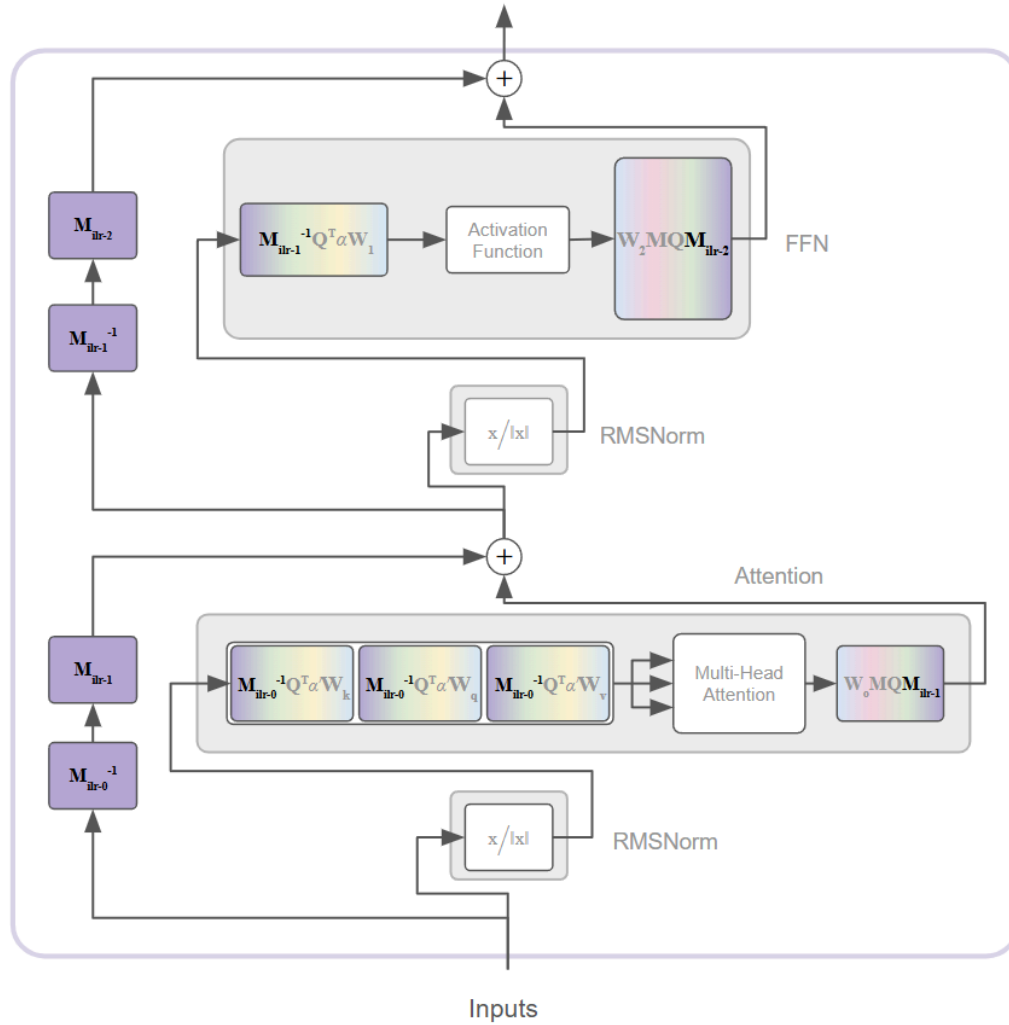


Figure 7 - Applying an I+LR transformation to a model.
Adapted from the SliceGPT paper.

2.4 Performing OATS compression

After applying the invariant transformations to the model, the final step is to apply the pruning procedure - in our case, we will be using OATS. For the OATS parameters, I used a rank ratio of 0.2, with two sparsity values: 0.3 and 0.5. These parameters were chosen to match those used to test the ViT model in the original OATS paper.

3.0 Results

To evaluate the effectiveness of our pruning methods compared with the regular OATS method, we will use a relative measure of effectiveness, referred to as recovery. For a given pruning method of interest, recovery refers to the proportion of lost performance from OATS pruning that was recovered using the pruning method of interest. A recovery value greater than 0 indicates that the accuracy from the new pruning method is an improvement over OATS (with the same OATS parameters), while a value less than 0 indicates that the accuracy of the new pruning method is worse than using regular OATS. Equation 2 shows the mathematical formulation of recovery.

Equation 2 - Accuracy recovery

$$R = \frac{a_{prune} - a_{reference}}{a_{base} - a_{reference}} = \frac{\text{orange}}{\text{blue}}$$

ACCURACY

a_{base} Accuracy of base model (with no pruning)

a_{prune} Accuracy of model of interest

$a_{reference}$ Accuracy of reference model (with OATS pruning using same OATS parameters as model of interest)

3.1 Effect of Rotating Model Weights

The first thing that I wanted to test was whether the base model is already in the optimal rotational orientation for OATS pruning. To test this, I rotated the model weights with a randomly generated rotation matrix before applying OATS, and compared the model accuracies with the non-rotated model. Figure 8 shows the absolute accuracy loss (compared to the non-pruned base model) on the left (a), the accuracy recovery of the randomly rotated model, with a sparsity of 0.3 at the center (b), and a sparsity of 0.5 on the right (c). On the x-axis, the top-n accuracy is shown from top-1 to top-10. In total, three randomly generated rotation matrices were tested.

For the lower OATS sparsity of 0.3 (the blue lines in the figure), the accuracies of the randomly rotated model are similar to the non-rotated model, with both positive and negative recoveries at different top-n. In contrast, at an OATS sparsity of 0.5 (the orange lines in the figure), the accuracy of the models with a random rotation is significantly lower than the non-rotated model. This suggests that, at least for lower OATS sparsity values, the base model is not necessarily in the optimal orientation for OATS pruning, and performance can potentially be improved through a rotation of the model weights.

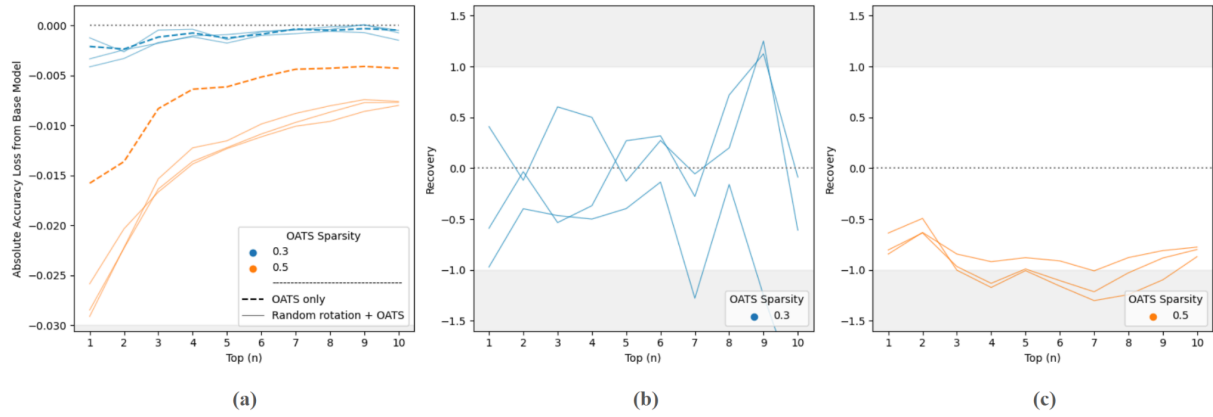


Figure 8 - Absolute accuracy loss relative to unpruned base model (a); Accuracy recovery of randomly rotated model with OATS sparsity of 0.3 (b) and 0.5 (c) from top-1 to top-10

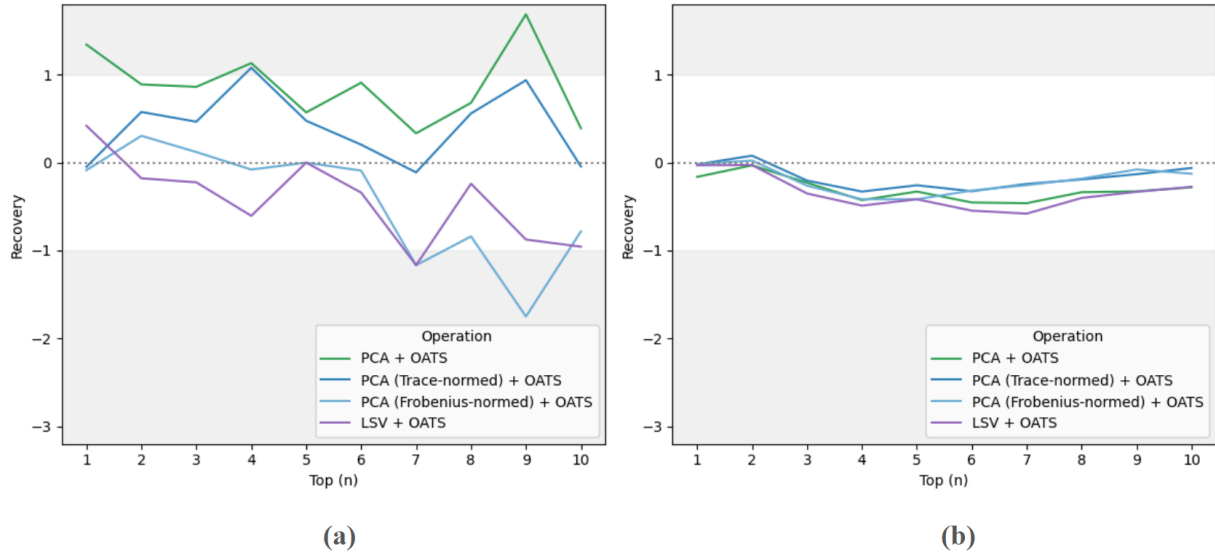


Figure 9 - Accuracy recovery of rotated model with OATS sparsity of 0.3 (a) and 0.5 (b) from top-1 to top-10

Random rotations to the model weights can result in a positive recovery by chance; however, using an intentionally constructed rotation matrix can further improve the results. Figure 9 shows the accuracy recovery of models after OATS pruning with various rotation matrices, using a sparsity of 0.3 on the left (a), and a sparsity of 0.5 on the right (b).

At the lower OATS sparsity of 0.3, the rotation matrix resulting in the highest recovery is PCA+OATS, which recovers nearly all of the accuracy lost with regular OATS pruning for many top-n values. Supplementing PCA with trace normalization actually worsens the performance, although the model still has a positive recovery at almost all top-n values tested. Meanwhile, using the Frobenius norm or LSV results in little improvement or even performance deterioration over regular OATS.

A brief explanation of how each rotation matrix is constructed is documented in Table 2.

Table 2 - Construction method of rotation matrices

Rotation Matrix	Method of construction
PCA + OATS	The rotation matrix is constructed from the principal components of all sampled embeddings from all layers (without any normalization operation)
PCA (Trace-normed) + OATS	The rotation matrix is constructed with the covariance matrix of sampled embeddings for each layer, normalized by their trace (as described in section 1.3.1)
PCA (Frobenius-normed) + OATS	The rotation matrix is constructed with the covariance matrix of sampled embeddings for each layer, normalized by their Frobenius norm (as described in section 1.3.1)
LSV + OATS	The rotation matrix is constructed by performing SVD on the rows of the model's weight matrices and using the left singular vectors (LSV)

Meanwhile, at the higher OATS sparsity of 0.5, all rotation matrices result in a performance deterioration for most top-n tested. At top-1 and top-2, applying a normalization step before doing PCA appears to have a slight improvement compared with no normalization, although the recovery is almost 0. Generally, the extent of recovery at the higher OATS sparsity of 0.5 appears to be more stable across all top-n.

3.2 Effect of Translation, Scale, and Permutation Transformations

Figure 10 shows the accuracy recovery after performing the five invariant transformations described in [section 1.3.2](#), with the recovery for the lower sparsity of 0.3 shown in the left (a) and the higher sparsity of 0.5 shown in the right (b).

At the lower sparsity of 0.3, the operation with the greatest improvement appears to be permutation, which has a performance improvement at nearly all top-n. Applying a translation operation improves performance in some top-n while deteriorating performance in others. Finally, the worst-performing operation is scaling, with a consistent loss in performance at all top-n. On the other hand, with a sparsity value of 0.5, none of the transformations improve the performance of the model.

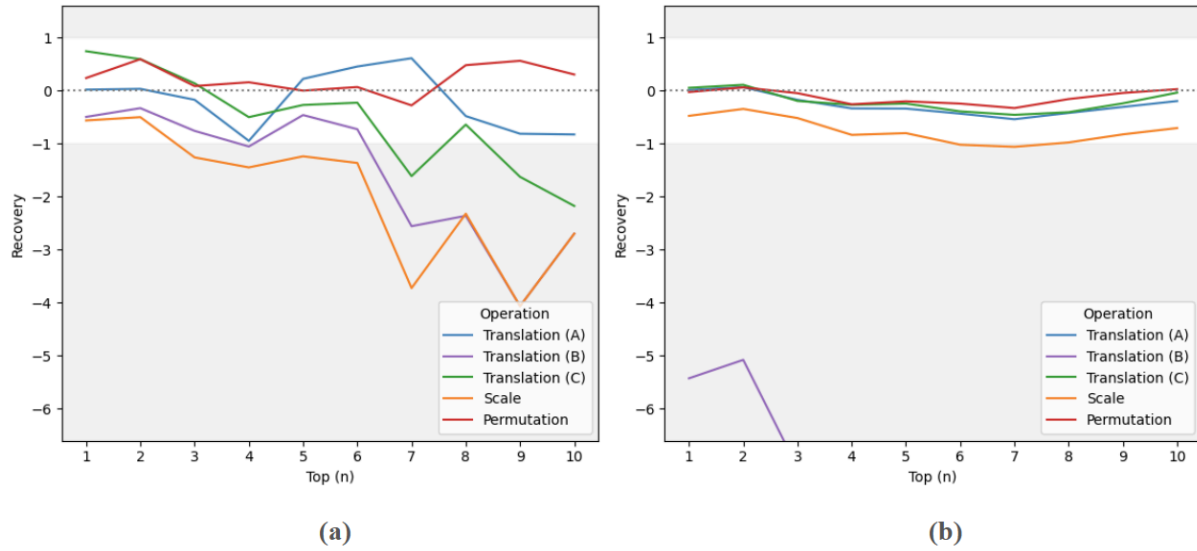


Figure 10 - Accuracy recovery of the model after applying various invariant transformations with OATS sparsity of 0.3 (a) and 0.5 (b) from top-1 to top-10

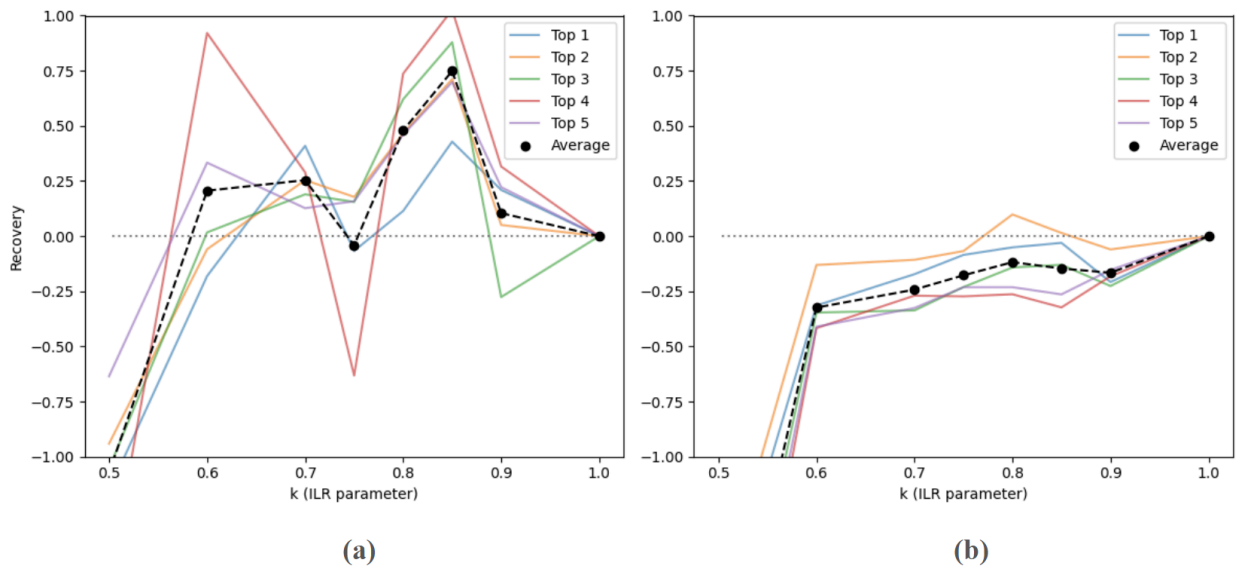


Figure 11 - Accuracy recovery of model after applying I+LR transformations with OATS sparsity of 0.3 (a) and 0.5 (b) at different k-values

Intuitively, some of these results make sense. For example, since outliers are important to model performance, it would make sense that normalizing the scale of the embeddings would degrade model performance, as they would attenuate the significance of outliers. However, other results are less clear. For example, it is not clear why the translation option B performs significantly worse than options A and C, especially since the same offset is applied in all methods.

3.4 Effect of Non-orthogonal Transformation

Consistent with the results thus far, applying a non-orthogonal transformation (as described in [section 1.3.3](#)) appears to potentially improve performance over regular OATS for the lower sparsity of 0.3, while degrading performance for the higher sparsity of 0.5. Figure 11 shows the recovery of the model with an I+LR transformation of rank 64, at a range of k-values on the x-axis from 0.5 to 1, with the lower sparsity of 0.3 on the left (a) and the higher sparsity of 0.5 on the right (b). Trendlines for top-1 to top-5 are graphed in colored lines, while the black trend shows the recovery, averaged from top-1 to top-5, at each k-value.

With a k value of 1, the ILR transformation effectively becomes an identity matrix (which does not modify the model weights), resulting in a recovery of 0. As k decreases to 0, the ILR construction approaches a rank-deficient matrix, which would consequently degrade performance.

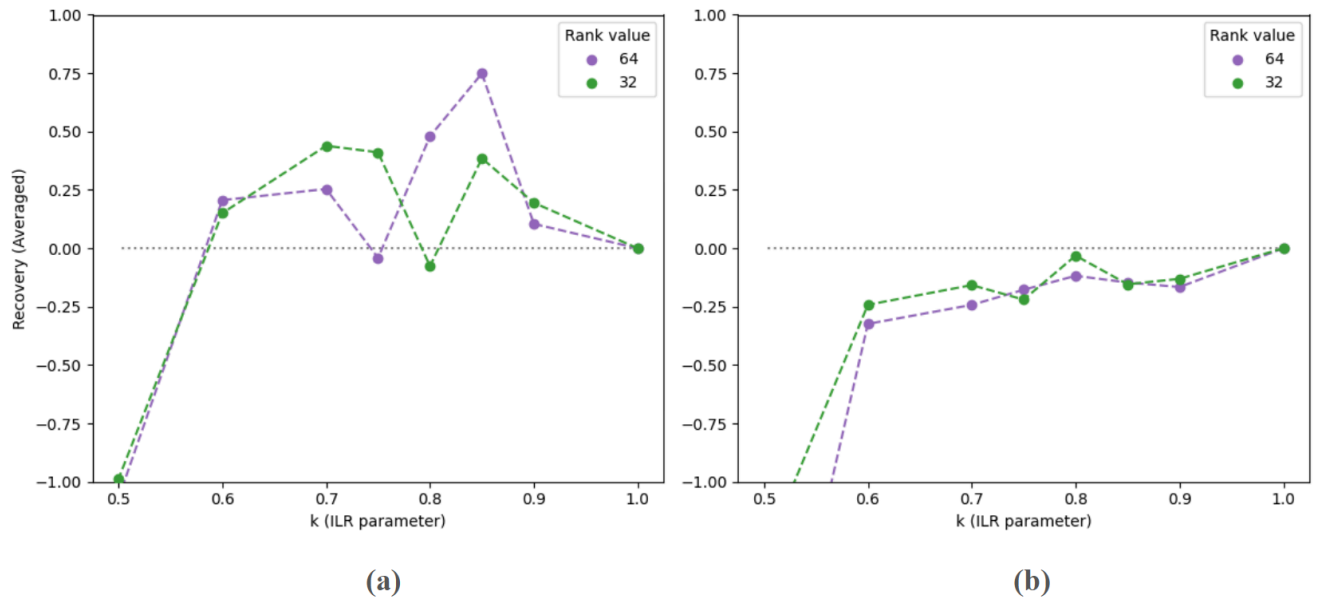


Figure 12 - Accuracy recovery of the model after applying rank-32 and rank-64 I+LR transformations with OATS sparsity of 0.3 (a) and 0.5 (b) at different k-values

Indeed, for the lower sparsity of 0.3, we can see that there is a value of k where the recovery peaks. For rank 64, the peak corresponds to a k -value of 0.85. Conversely, no peak is observed for the higher sparsity of 0.5; applying a transformation with any k -value less than 1 degrades performance.

The optimal k -value is also dependent on the rank of the low-rank component of the I+LR construction. For example, Figure 12 shows the recovery, averaged from top-1 to top-5, of a rank 64 construction (purple), compared to rank 32 (green). For the rank 64 construction, the recovery peaks at a k -value around 0.85, compared to a k -value of 0.7 at rank 32.

As mentioned in [section 1.3.3](#), the residual stream needs to be transformed with the I+LR transformation at each layer, incurring a small computational and memory cost. A rank 64 I+LR construction for the Google ViT model corresponds to a 2.6% increase in model parameters, and rank 32, 1.3%.

4.0 Discussion

The results from section 3 suggest that it is possible to modify a model using invariant transformations to mitigate the accuracy loss of OATS pruning. However, this only appears to be the case for lower OATS sparsity values. At a higher sparsity of 0.5, all transformations explored ended up worsening performance. Additionally, at the lower sparsity of 0.3, the accuracy loss from OATS pruning is quite small (0.2%). In this case, although we can minimize the loss using invariant transformations, the total accuracy gain is relatively minimal.

Thus far, the focus has been to complement the OATS pruning process by applying invariant transformations to the model. The theoretical basis for many of the design and implementation decisions that I made is heavily inspired by the process of PCA; however, it is unclear whether PCA produces the optimal transformations that maximally improve OATS performance. Future methods can explore alternative methods of constructing transformations, for example, by using an iterative process to find optimal transformations. In addition, future work can be done to evaluate if invariant transformations can improve other pruning methods, such as SparseGPT [14], to a greater extent than OATS, or if similar results can be seen on different models or LLMs.

References

- [1] Q. Chen, L. Qin, J. Liu, D. Peng, J. Guan, P. Wang, M. Hu, Y. Zhou, T. Gao, W. Che. *Towards Reasoning Era: A Survey of Long Chain-of-Thought for Reasoning Large Language Models*. arXiv:2503.09567 [cs.AI], March 2025
- [2] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso. *The Computational Limits of Deep Learning*. arXiv:2007.05558 [cs.LG], July 2020

- [3] E. Frantar, and D. Alistarh. *SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot*. arXiv:2301.00774 [cs.LG], January 2023
- [4] C. Hu, X. Li, D. Liu, H. Wu, X. Chen, J. Wang, X. Liu. *Teacher-Student Architecture for Knowledge Distillation: A Survey*. arXiv:2308.04268 [cs.LG], August 2023
- [5] V. Papyan, X. Y. Han, and D. L. Donoho. *Prevalence of neural collapse during the terminal phase of deep learning training*. arXiv:2008.08186 [cs.LG], August 2020
- [6] S. Noroozizadeh, V. Nagarajan, E. Rosenfeld, and S. Kumar. *Deep sequence models tend to memorize geometrically; it is unclear why*. arXiv:2510.26745 [cs.LG], October 2025
- [7] J. Engles, E. J. Michaud, I. Liao, W. Gurnee, and M. Tegmark. *Not All Language Model Features Are One-Dimensionally Linear*. arXiv:2405.14860 [cs.LG], May 2024
- [8] Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. *The Clock and the Pizza: Two Stories in Mechanistic Explanation of Neural Networks*. arXiv:2306.17844 [cs.LG], November 2023
- [9] J. Lindsey, W. Gurnee, E. Ameisen, B. Chen, A. Pierce, N. L. Turner, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. B. Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, J. Batson. *On the Biology of a Large Language Model*. Anthropic. March 2025
- [10] B. Wu, C. Xu, X. Dai, A. Wan, P. Zhang, Z. Yan, M. Tomizuka, J. Gonzalez, K. Keutzer, and P. Vajda. *Visual Transformers: Token-based Image Representation and Processing for Computer Vision*. arXiv:2006.03677 [cs.CV], 2020
- [11] S. Zhang and V. Papyan, *OATS: Outlier-Aware Pruning Through Sparse and Low Rank Decomposition*. arXiv:2409.13652 [cs.LG], September 2024.
- [12] S. Ashkboos, M. L. Croci, M. G. D. Nascimento, T. Hoefler, and J. Hensman, *SliceGPT: Compress Large Language Models by Deleting Rows and Columns*. arXiv:2401.15024 [cs.LG], January 2024.
- [13] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, *QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs*. arXiv:2404.00456 [cs.LG], March 2024.
- [14] E. Frantar, and D. Alistarh, *SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot*. arXiv:2301.00774 [cs.LG], January 2023